

Public Agent Skills Are Becoming the Next RL Dataset

A Framework for Extracting Verified Training Signal from Procedural Skill Definitions

Authors: 4M Labs Research

Affiliation: 4M Labs

Date: July 2026

Status: Working Paper (Pre-print, not peer-reviewed)

Corresponding Author: hello@4mlabs.io

Cite as: 4M Labs Research. "Public Agent Skills Are Becoming the Next RL Dataset." Working Paper, July 2026.

Abstract

A structural convergence is underway in the open-source AI ecosystem. Thousands of structured agent skill definitions, files named SKILL.md hosted on GitHub repositories, are accumulating across communities. Each encodes a procedural policy: step-by-step instructions for accomplishing a task, optionally bundled with evaluation scripts, assertions, grading logic, and benchmark infrastructure.

We observe that these skills can be converted into verified reinforcement learning environments through a simple comparative method: generate evaluation tasks from the skill itself, run two agents on identical tasks (one with the skill, one without), score both using the generated evaluations, and use the performance delta as training signal. The better-performing outputs become preference pairs for DPO and GRPO training.

This paper describes SkillGym, a framework for ingesting arbitrary SKILL.md files, generating evals, running controlled comparisons, and producing structured datasets for supervised fine-tuning (SFT), direct preference optimization (DPO), and group relative policy optimization (GRPO). We discuss which domains benefit most, trust requirements, and why narrow procedural competence is a natural fit for small models. Experimental validation is planned.

1. Introduction: Skills as Procedural Policies

The agent skills ecosystem emerged from a practical problem: large language models are capable but unfocused. They can write code, but they do not know your deployment process. They can analyze data, but they do not know your schema conventions. Skills solve this by injecting domain-specific procedural knowledge at inference time.

A typical SKILL.md file contains structured instructions (what to do and in what order), optionally bundled with scripts (deterministic operations), evaluation criteria (how to tell if the output is correct), and references (domain documentation loaded on demand).

Two distinct research directions have emerged around skills:

1. Skill optimization: Improving the skill itself. Microsoft's SkillOpt (Yang et al., 2026) treats the skill document as a trainable parameter, using a stronger optimizer model to propose bounded text edits validated against held-out tasks. The target model stays frozen. The skill gets better.

2. Skill as training data: Using skills to improve the model itself. SkillGym takes the opposite approach. The skill is not optimized. Instead, the skill becomes the verifier: a source of structured evaluation criteria against which model outputs are scored. The delta between performance with and without the skill becomes training signal for model fine-tuning.

These approaches are complementary. SkillOpt produces better skills. SkillGym uses those skills to produce better models. A skill optimized by SkillOpt, when used as input to SkillGym, would generate higher-quality training data because the skill's instructions are sharper and its evaluation criteria are more precise.

This paper focuses on the second direction: using skills as verified training environments.

2. The Comparative Method

The core insight is simple: you do not need skills that come with evals. You can generate evals from the skill itself, then run a controlled experiment.

2.1 The Pipeline

Step 1: Ingest. Pull a SKILL.md from any public GitHub repository. Parse the instructions, references, scripts, and workflow descriptions.

Step 2: Generate Evals. Use the skill's own content to generate evaluation tasks. The skill describes what it does. The eval generator produces concrete tasks that test whether those descriptions work. Each eval includes a prompt, optional input files, and expected outcome criteria.

Step 3: Run Comparison. Spin two sub-agents on the same eval tasks:

Agent A: receives the skill context (SKILL.md, references, scripts)

Agent B: receives no skill context (baseline)

Both agents attempt the same tasks independently.

Step 4: Score. Run the generated evals against both outputs. The evals check: Does the output contain required fields? Does the code compile? Does the schema validate? Does the deployment succeed?

Step 5: Compute Delta. For each eval task:

Pass rate with skill (Agent A)

Pass rate without skill (Agent B)

Delta = Agent A pass rate minus Agent B pass rate

Step 6: Export Training Data. Use the delta to generate:

- SFT traces from successful skill-assisted outputs
- DPO pairs from tasks where Agent A passed and Agent B failed
- GRPO groups from multiple attempts at the same task

2.2 Why This Works

The skill's own instructions define what "good" looks like. The evals test whether the model achieved it. The comparison isolates the skill's contribution: if the model performs better with the skill than without it, the skill is providing real procedural guidance, and the outputs that benefit from it are verified training examples.

This is not synthetic data generation. The model is not generating its own reward signal. The evals are generated from the skill's structure, not from a judge model's preferences. The comparison is controlled: same model, same tasks, different context.

2.3 What the Delta Represents

The delta is the measurable value of the skill. A delta of 0 means the skill provides no benefit for that task type. A positive delta means the skill improves performance. A negative delta means the skill confuses the model.

For training purposes, the interesting cases are:

- Tasks where Agent A passes and Agent B fails: these become DPO chosen/rejected pairs
- Tasks where both pass: these become SFT training examples (the skill provides clean procedure)
- Tasks where both fail: these are discarded or used for curriculum filtering
- Tasks where Agent A fails and Agent B passes: the skill is harmful for this task type

2.4 The Skill Is the Verifier

The critical distinction: we do not build new verifiers. The skill's own structure provides the verification criteria. When a SKILL.md says "the output must include a SUM formula in cell B10" or "the API must return a 200 status code," those become eval assertions. The skill author already defined what success looks like. We extract that definition and use it to score model outputs.

This means the pipeline scales with the number of skills, not with the number of verifiers we build. Every new skill brings its own evaluation criteria.

3. Training Data Export

3.1 SFT: Learn from Successful Traces

Filter to tasks where Agent A (with skill) passes all eval criteria. Export as:

```
{
  "instruction": "eval prompt",
  "context": "skill instructions and references",
  "trajectory": "agent output (tool calls, files created, commands run)",
  "reward": 1.0
}
```

These examples teach the model to follow the skill's procedure successfully. SFT is the foundation: before preference optimization, the model needs to see what good performance looks like.

When to use SFT alone:

- The skill has high delta (large performance improvement with skill)
- The eval criteria are comprehensive (most success conditions are covered)
- The task domain is narrow and procedural (code, schemas, documents)
- You want to bootstrap a small model quickly without RL complexity

3.2 DPO: Learn from Preference Pairs

Filter to tasks where Agent A passes and Agent B fails. Export as:

```
{
  "prompt": "eval prompt",
  "context": "skill instructions",
  "chosen": "Agent A output (passed)",
  "rejected": "Agent B output (failed)",
  "chosen_reward": 1.0,
  "rejected_reward": 0.0
}
```

The preference is grounded in eval pass/fail, not model judgment.

When to use DPO:

- You have enough pass/fail pairs (minimum hundreds, ideally thousands)
- The delta is moderate (skill helps but not always)
- You want to refine behavior without full RL infrastructure
- The eval criteria are binary (pass/fail, not partial credit)

3.3 GRPO: Learn from Grouped Attempts

For each eval prompt, run Agent A multiple times with different temperatures or seeds. Score each attempt against the evals. Export as:

```
{
  "prompt": "eval prompt",
  "context": "skill instructions",
  "group": [
    {"completion": "attempt 1", "reward": 0.0},
    {"completion": "attempt 2", "reward": 0.6},
    {"completion": "attempt 3", "reward": 1.0},
    {"completion": "attempt 4", "reward": 0.0}
  ]
}
```

The model learns from relative success within a group, without requiring a critic.

When to use GRPO:

The eval criteria support partial credit (not just binary pass/fail)

You can sample many attempts per task (budget allows multiple rollouts)

You want the model to learn from its own distribution shift

DPO pairs are noisy or insufficient

3.4 Choosing the Right Method

Signal Type	Method	Minimum Data	Best For
High delta, comprehensive evals	SFT	Hundreds of examples	Quick bootstrap, narrow domains
Moderate delta, binary evals	DPO	Thousands of pairs	Refinement without RL
Partial credit, many attempts	GRPO	Many groups per task	Continuous improvement

In practice, a pipeline would run SFT first, then DPO, then GRPO iteratively. Each stage builds on the previous one.

4. Domains That Benefit Most

4.1 Strong Domains

Code generation: Code either compiles or it does not. Unit tests verify behavior. Skills for code review, refactoring, or test generation produce verifiable outputs.

Schema and data operations: JSON/YAML schemas validate or reject. Database migrations apply cleanly or break. Data transformations preserve invariants or corrupt them.

Document generation: Invoices have required fields. Proposals have structural requirements. Contracts have clause completeness. Skills for business documents can verify output structure and content.

Deployment and operations: Health checks pass or fail. Services start or crash. Operations skills have the strongest verification because infrastructure provides ground truth.

API integration: API calls succeed or fail with status codes. Request/response schemas validate. Rate limits are respected or violated.

4.2 Weak Domains

Creative writing, strategy, UX design: Quality is subjective. Automated evals can check structure but not artistic merit or strategic value. These are poor candidates for this method.

5. Trust and Sandboxing

5.1 The Problem

Skills can contain arbitrary instructions. A malicious or poorly written skill could include prompt injection attempts, request data exfiltration, instruct destructive operations, or include fake evals that reward harmful outputs.

5.2 Mitigation

Skill scanning: Static analysis of SKILL.md for suspicious patterns before ingestion.

Evals generated independently: The eval generator does not trust the skill's self-assessment.

Sandboxed execution: Agent runs in isolated environments with network restrictions.

Output verification: Generated evals are checked for fairness and completeness.

Delta sanity checks: Skills with suspiciously high or low deltas are flagged for review.

6. Small Model Advantage

Small models (1B-8B parameters) cannot compete with frontier models on general intelligence. The better strategy is narrow procedural competence.

A 3B model fine-tuned on verified skill data can follow specific deployment procedures accurately, generate correct API integrations for known services, apply consistent formatting rules, and execute multi-step workflows reliably.

The economics work: compile data from public skills for near-zero cost, fine-tune on a single GPU in hours, serve at a fraction of general-purpose API costs.

7. Related Work

7.1 Skill Optimization

SkillOpt (Yang et al., 2026): Microsoft's approach to optimizing skills through text-space edits. A stronger optimizer model proposes bounded add/delete/replace edits to the skill document, validated against held-out tasks. The target model stays frozen. SkillOpt achieves +23.5 points average improvement on GPT-5.5 across six benchmarks. The optimized skill transfers across model scales and execution harnesses.

SkillGym is complementary: SkillOpt makes better skills, SkillGym uses those skills to make better models. An optimized skill produces higher-quality training data because its instructions are sharper and its evaluation criteria are more precise.

EvoSkill (anonymous, 2026): Evolutionary approach to skill generation through self-revision. Less controlled than SkillOpt.

Trace2Skill (anonymous, 2026): Extracts skills from agent execution traces. Produces skills from observed behavior rather than hand-authoring.

7.2 Skill and Tool Learning

Toolformer (Schick et al., 2023), **Gorilla** (Chen et al., 2023), and **ToolLLM** (Qin et al., 2023) teach models to call external APIs. Skills encode broader procedural knowledge beyond single API calls.

7.3 Self-Improvement and Self-Play

SPIN (Chen et al., 2024) and **Self-Rewarding Language Models** (Yuan et al., 2024) use model outputs as training signal. SkillGym differs by using skill-generated evals and controlled comparisons rather than self-evaluation.

7.4 Constitutional AI and RLAI

Constitutional AI (Bai et al., 2022) uses AI-generated feedback. SkillGym uses evals derived from skill structure, not from a judge model. The verification is grounded in task requirements, not model preferences.

7.5 Code Generation Benchmarks

HumanEval (Chen et al., 2021) and **SWE-bench** (Jimenez et al., 2024) verify code execution. SkillGym extends verification to any domain with procedural structure.

8. Future Work

Proof of concept: Implement the full pipeline for 10-20 skills across code, schema, and document generation domains. Measure delta distribution and eval quality.

Training experiments: Fine-tune 1B-3B models on DPO pairs from skill comparisons. Compare against baseline models on held-out evals.

Integration with SkillOpt: Test whether SkillOpt-optimized skills produce higher-quality training data than raw skills. Measure the delta difference between optimized and unoptimized skills.

Ecosystem survey: Catalog skills suitable for comparison-based training. Estimate total verifiable training data volume.

Open questions: Does the comparison method produce cleaner signal than synthetic data? Can small models trained on skill data compete with larger models on procedural tasks? What is the minimum delta threshold for useful training signal? Do optimized skills (via SkillOpt) produce proportionally better training data?

9. Conclusion

The agent skills ecosystem has accumulated something valuable: procedural policies with evaluable structure. SkillGym proposes a simple method to extract training signal from them: generate evals, run controlled comparisons, use the delta as reward.

The curriculum is already written by thousands of independent contributors. The verification comes from the skill's own structure. The comparison isolates the skill's contribution. The resulting data is grounded in actual task completion.

Two paths forward exist for skills: optimize the skill (SkillOpt) or use the skill to optimize the model (SkillGym). These are not competing approaches. They are complementary stages in a pipeline where better skills produce better training data, which produces better models, which execute skills more competently.

If validated, this would offer a cost-effective path to training specialized models: compile skill data, train small models, evaluate against the same evals, and iterate.

References

1. Anthropic. "Skill Creator: A Skill for Creating and Iterating on Agent Skills." GitHub, 2026.
2. Bai, Y. et al. "Constitutional AI: Harmlessness from AI Feedback." arXiv:2212.08073, 2022.
3. Chen, M. et al. "Evaluating Large Language Models Trained on Code." arXiv:2107.03374, 2021.
4. Chen, X. et al. "Gorilla: Large Language Model Connected with Massive APIs." arXiv:2305.15334, 2023.
5. Chen, Z. et al. "SPIN: Self-Play Fine-Tuning Converts Weak Language Models to Strong Language Models." ICML, 2024.
6. DeepSeek-AI. "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning." arXiv, 2025.
7. Jimenez, C. E. et al. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" ICLR, 2024.
8. Li, Y. et al. "Direct Preference Optimization: Your Language Model is Secretly a Reward Model." NeurIPS, 2023.
9. Qin, Y. et al. "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs." arXiv:2307.16789, 2023.
10. Shao, Z. et al. "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models." arXiv, 2024.
11. Schick, T. et al. "Toolformer: Language Models Can Teach Themselves to Use Tools." arXiv:2302.04761, 2023.
12. Yang, Y. et al. "SkillOpt: Executive Strategy for Self-Evolving Agent Skills." arXiv:2605.23904, 2026.
13. Yuan, W. et al. "Self-Rewarding Language Models." arXiv:2401.10020, 2024.